

COURSE INTRO

생성형 AI를 활용한 챗봇 만들기

1일 8시간 집중 과정

과정 안내

본 과정은 1일 8시간 집중 과정으로, OpenAI API 기초부터 RAG 기반 챗봇 구현까지 전 과정을 다룹니다.

과정 정보

과정명 생성형 AI를 활용한 챗봇 만들기

시간 1일 8시간 (09:00 ~ 18:00)

강사 조나래

날짜 2026.03.20

강사 소개

이름 조나래

소속 주식회사 드림플로우 (대표)

이메일 info@dream-flow.com

전문 분야 AI/LLM 챗봇

학력 가천대학교 공학석사

경력 스타트업 플랫폼 운영 3년 · KOSTA 재직자 과정 5년

학습 목표

오늘 워크샵을 마치면 다음을 할 수 있습니다

- 생성형 AI와 LLM의 기본 개념을 설명할 수 있습니다
- OpenAI API로 단일턴/멀티턴 대화를 구현할 수 있습니다
- 프롬프트 엔지니어링 기법(Zero/One/Few-shot)을 적용할 수 있습니다
- Streamlit으로 웹 기반 챗봇을 구축할 수 있습니다
- LangChain, LCEL, Tool/Agent를 활용할 수 있습니다
- 시스템 프롬프트의 한계를 이해하고 RAG의 필요성을 설명할 수 있습니다
- RAG 파이프라인(로딩, 청킹, 임베딩, 벡터DB, 체인)을 구현할 수 있습니다

오늘 만들 것

API 호출부터 RAG 챗봇까지, 단계적으로 기능을 쌓아 갑니다



OpenAI API 챗봇

API 호출로 AI와 대화하는 기본 챗봇



Streamlit 웹 챗봇

브라우저에서 동작하는 채팅 인터페이스



LangChain Agent

도구를 자동 선택하여 문제를 해결하는 AI



RAG 챗봇 (최종)

PDF 문서를 검색하여 근거 기반 답변을 생성하는 챗봇

타임테이블

오전에는 API와 프롬프트 기초를, 오후에는 LangChain과 RAG를 다룹니다

시간	내용	비고
09:00 ~ 10:00	OT, 생성형 AI 소개, 환경설정	블록 1
10:00 ~ 11:00	OpenAI API 기초, 멀티턴 대화	블록 2
11:00 ~ 11:50	프롬프트 엔지니어링 (Zero/One/Few-shot)	블록 3
11:50 ~ 13:00	점심	
13:00 ~ 14:00	Streamlit 웹 챗봇, LangChain 입문	블록 4
14:00 ~ 15:00	LangChain 활용, LCEL 체인 구성	블록 5
15:00 ~ 16:00	LCEL 심화, Tool과 Agent	블록 6
16:00 ~ 17:00	고객상담 챗봇, 한계 체험, RAG 이론	블록 7
17:00 ~ 18:00	RAG 파이프라인 구현, 실전 예제, 마무리	블록 8

환경설정

워크샵에 필요한 개발 환경을 준비합니다

- Python 3.10 이상 설치 확인
- UV 패키지 매니저 설치
- 프로젝트 클론 및 의존성 설치
- .env 파일에 OpenAI API 키 설정
- 첫 API 테스트 실행

```
환경설정 순서 bash

# 1. Python 3.10+ 확인
python --version

# 2. UV 패키지 매니저 설치
curl -LsSf https://astral.sh/uv/install.sh | sh

# 3. 의존성 설치
cd chatbot-private && uv sync

# 4. .env 파일 생성
echo "OPENAI_API_KEY=sk- ..." > .env

# 5. 첫 테스트
uv run python 01.settings/01.test.py
```

01

생성형 AI 이해

- ✓ AI, ML, DL, 생성형 AI의 관계를 계층 구조로 설명할 수 있습니다
- ✓ LLM의 동작 원리를 '다음 토큰 예측'으로 이해합니다
- ✓ 토큰, Temperature 개념과 주요 AI 모델을 파악합니다

🕒 25분

AI의 계층 구조: 큰 그림 이해하기

마트료시카(러시아 인형)처럼, 큰 개념 안에 작은 개념이 포함됩니다



LLM 동작 원리: "다음 토큰 예측"

인터넷의 수십억 페이지 텍스트를 학습한 AI가 다음에 올 단어를 예측합니다

- 핵심 원리: "지금까지 나온 단어를 보고, 다음에 올 단어를 예측"합니다
- 스마트폰 자동완성의 극한 버전이라고 이해하면 됩니다
- 수천 단어의 맥락을 한 번에 참고하여 문맥과 의미를 종합 판단합니다
- 이 과정을 반복하면 문장, 문단, 글 전체가 완성됩니다

다음 토큰 예측 예시

입력: "오늘 날씨가 정말"

예측 후보:

"좋습니다" (확률 45%)

"덥습니다" (확률 20%)

"춥습니다" (확률 15%)

"맑습니다" (확률 12%)

토큰: AI가 글을 읽는 단위

같은 의미라도 한국어는 영어보다 토큰을 약 1.5배에서 2배 더 사용합니다

텍스트	언어	토큰 수	토큰 분리 예시
"Hello, world!"	영어	약 3개	Hello, ,, world!
"안녕하세요"	한국어	약 3-4개	안녕, 하, 세요
"I love programming"	영어	약 3개	I, love, programming
"나는 프로그래밍을 좋아합니다"	한국어	약 7-8개	나는, 프로그래, 밍을, 좋아, 합니다

항목	설명
비용	API 사용료가 토큰 수 기준으로 부과됩니다
입력 한도	한 번에 보낼 수 있는 텍스트 양에 제한이 있습니다
속도	토큰이 많을수록 응답 생성 시간이 길어집니다

직접 확인해 봅시다

OpenAI Tokenizer에서 텍스트를 입력하면 토큰이 어떻게 분리되는지 시각적으로 확인할 수 있습니다.

platform.openai.com/tokenizer

한국어 문장을 입력하여 영어 대비 토큰 수 차이를 직접 비교합니다

Temperature: 응답의 성격을 조절하는 다이얼

0.0(일관적)부터 1.0(창의적)까지, 용도에 따라 조절합니다

Temperature	요리사 비유	특징
0.0	레시피를 글자 그대로 따르는 요리사	매번 동일한 결과, 예측 가능
0.7	레시피를 참고하되 자기 스타일을 가미	균형 잡힌 창의성
1.0	레시피를 영감 삼아 즉흥 요리하는 셰프	높은 다양성, 뜻밖의 결과

용도	권장 Temperature	이유
코드 생성, 데이터 추출	0.0 ~ 0.2	정확성이 최우선
일반 질의응답, 요약	0.3 ~ 0.5	정확하면서 자연스러운 표현
이메일 작성, 고객 응대	0.5 ~ 0.7	자연스러운 대화체
창작, 브레인스토밍	0.7 ~ 1.0	다양한 아이디어 필요

질문: "서울을 한 문장으로 소개해주세요"

0.0 결정론적, 매번 같은 답

서울은 대한민국의 수도이자 약 950만 명이 거주하는 정치, 경제, 문화의 중심지입니다.

0.7 균형, 적당한 변화

서울은 600년 역사를 품은 대한민국의 수도로, 전통과 현대가 조화를 이루는 도시입니다.

1.0 창의적, 다양한 답

천만 개의 이야기가 한강 위를 흐르는 곳, 그곳이 서울입니다.

주요 생성형 AI 모델 비교

2026년 3월 기준, 각 분야의 최신 모델입니다

모델	개발사	특징	주요 용도
GPT-5.4	OpenAI	추론 모드(Thinking) 탑재, 최신 플래그십	범용 대화, 고급 추론
GPT-4.1 Nano	OpenAI	경량 모델, 빠르고 저렴 (오늘 사용)	실시간 응답, 학습용
Claude Sonnet 4.6	Anthropic	속도와 지능의 균형, 토큰 효율성 개선	코딩, 문서 분석
Gemini 2.5 Pro	Google	코딩 1위, 네이티브 음성 출력	멀티모달, 대용량 문서
Llama 4 Maverick	Meta	오픈소스, 128 전문가 혼합, GPT-4o 초과	자체 서버 운영, 커스텀

분야	대표 도구	특징
이미지 생성	GPT Image 1.5, Midjourney v7, FLUX.1.1	텍스트로 이미지 생성, 사실성 향상
코드 생성	Claude Code (1위), Cursor, GitHub Copilot	자율 에이전트, 1M 토큰 컨텍스트
음성 생성	ElevenLabs, Suno v5	음성 클로닝, 가사 포함 작곡
영상 생성	Sora 2, Kling	물리학 정확성, iOS 앱 출시

생성형 AI 활용 사례

이미 다양한 분야에서 실무에 활용되고 있습니다



텍스트 생성

보고서 초안, 이메일 작성, 회의록 요약, 실시간 다국어 번역, 24시간 자동 응답 챗봇 구축에 활용됩니다.



이미지 생성

마케팅 배너, 제품 목업, 프로토타입 디자인, 슬라이드용 일러스트 자동 생성에 활용됩니다.



코드 생성

코드 작성, 리뷰, 버그 수정, 데이터 시각화, SQL 쿼리 생성 등 개발 생산성을 높입니다.



음성 및 영상

나레이션, 팟캐스트 더빙, 제품 소개 영상, 광고 콘텐츠, 음악 작곡 등에 활용됩니다.

오늘의 여정: 8시간 워크샵 로드맵

"Hello, API"부터 시작하여 완전한 RAG 챗봇까지 단계적으로 만들어 갑니다

- ▶ 블록 1 (09:00~10:00): OT, 생성형 AI 소개, 환경설정 — Python 환경 준비, API 연결 확인
- ▶ 블록 2 (10:00~11:00): OpenAI API 기초 — API로 AI와 대화하기, 멀티턴 대화 구현
- ▶ 블록 3 (11:00~11:50): 프롬프트 엔지니어링 — Zero/One/Few-shot 기법으로 답변 품질 향상
- ▶ 블록 4 (13:00~14:00): Streamlit 웹 챗봇, LangChain 입문 — 웹 브라우저 챗봇
- ▶ 블록 5 (14:00~15:00): LCEL 체인 구성 — 여러 단계를 연결한 AI 파이프라인
- ▶ 블록 6 (15:00~16:00): Tool과 Agent — AI가 도구를 선택하여 문제 해결
- ▶ 블록 7 (16:00~17:00): 고객상담 챗봇 — 시스템 프롬프트 한계 체험, RAG 필요성
- ▶ 블록 8 (17:00~18:00): RAG 파이프라인 구현 — PDF 문서 기반 RAG 챗봇 완성

체크포인트

- ✓ AI, ML, DL, 생성형 AI의 관계를 마트료시카 비유로 설명할 수 있습니까?
- ✓ LLM이 '다음 토큰 예측'으로 동작한다는 원리를 이해했습니까?
- ✓ 한국어가 영어보다 토큰을 더 많이 사용하는 이유와 그 영향을 설명할 수 있습니까?
- ✓ Temperature 0.0과 1.0의 차이를 요리사 비유로 설명할 수 있습니까?

02

OpenAI API와 대화 관리

- ✓ API 호출 구조(model, messages, temperature)를 이해합니다
- ✓ system, user, assistant 3가지 역할의 차이를 설명할 수 있습니다
- ✓ 멀티턴 대화에서 히스토리 관리의 원리를 파악합니다

🕒 50분

API 호출 구조

편지 보내기 비유: 봉투(파라미터)에 편지지(messages)를 넣어 보냅니다

기본 API 호출

python

```
from openai import OpenAI

client = OpenAI() # 환경변수에서 키 자동 로드

response = client.chat.completions.create(
    model="gpt-4.1-nano",
    messages=[
        {"role": "system",
         "content": "당신은 친절한 도우미입니다"},
        {"role": "user",
         "content": "안녕하세요"}
    ],
    temperature=0.7,
    max_tokens=500,
)

# AI의 답변 꺼내기
answer = response.choices[0].message.content
```

파라미터	필수	설명
model	필수	사용할 AI 모델 이름 (gpt-4.1-nano)
messages	필수	대화 내용 배열. role과 content로 구성
temperature	선택	응답 창의성 조절 (0.0에서 2.0)
max_tokens	선택	응답 최대 길이 (토큰 단위)

메시지 유형 3가지

OpenAI API의 모든 대화는 system, user, assistant 세 가지 역할로 구성됩니다

system 개발자가 작성

시스템 메시지

AI의 성격, 규칙, 응답 형식을 설정합니다

신입사원에게 주는 업무 매뉴얼과 같습니다. 처음 한 번 전달하면 이후 모든 대화에 적용됩니다.

"당신은 Python 전문가입니다. 초보자에게 친절하게 설명합니다."

user 사용자가 입력

사용자 메시지

질문, 요청, 지시 등 사용자의 입력을 담습니다

고객이 상담사에게 하는 질문입니다. 대화가 이어질 때마다 새로운 user 메시지가 추가됩니다.

"딕셔너리가 뭐야?"

assistant AI의 응답 (기록용)

어시스턴트 메시지

AI가 이전에 답변한 내용을 기록하여 맥락을 유지합니다

상담 일지에 기록한 이전 답변입니다. 다음 대화 때 이것을 함께 보내야 AI가 자신이 뭐라고 했는지 압니다.

"딕셔너리는 키-값 쌍으로 데이터를 저장하는 자료형입니다..."

싱글턴과 멀티턴

대화 방식의 두 가지 유형을 구분합니다

싱글턴 (Single-turn)

질문 하나, 답변 하나로 끝나는 1회성 대화입니다. 이전 대화를 기억하지 못합니다.

예시: 번역기, 요약기, 분류기, 1회성 질의응답

멀티턴 (Multi-turn)

여러 차례 이어지는 대화입니다. 이전 대화를 messages 배열에 누적하여 맥락을 유지합니다.

예시: 챗봇, 튜터, 상담 시스템, 대화형 에이전트

핵심 차이

항목	싱글턴	멀티턴
대화 횟수	1회 (질문 1개, 답변 1개)	여러 회 (대화가 이어짐)
이전 대화	기억하지 못합니다	messages에 누적하여 기억합니다
messages 크기	항상 1~2개	대화할수록 커집니다
구현 핵심	API 1회 호출	messages.append로 히스토리 누적
비용	매 호출 일정	대화가 길어질수록 증가

API는 stateless입니다. 멀티턴은 "API가 기억하는 것"이 아니라 "개발자가 매번 전체 대화를 보내는 것"입니다.

싱글턴 vs 멀티턴: 실제 동작 비교

같은 질문 "내 이름이 뭐라고 했지?"에 대한 결과가 완전히 달라집니다

싱글턴 (히스토리 없음)

AI
안녕하세요, 민수님!

내 이름은 민수야

AI
죄송합니다, 아직 이름을 알려주시지 않았는데요.

내 이름이 뭐라고 했지?

매번 **messages**를 새로 만들어서 맥락이 끊깁니다

멀티턴 (히스토리 누적)

AI
안녕하세요, 민수님!

내 이름은 민수야

AI
민수님이라고 하셨습니다!

내 이름이 뭐라고 했지?

이전 대화를 **messages**에 누적하여 맥락을 유지합니다

messages 배열은 이렇게 커집니다

매 턴마다 이전 대화 전체를 보냅니다. 노란색이 새로 추가된 부분입니다.

1 1턴째

user만 보냅니다

```
messages = [  
  {  
    "role": "system",  
    "content": "Python 전문가"  
  },  
  {  
    "role": "user",  
    "content": "딕셔너리가 뭐야?"  
  }  
]
```

배열 크기: 2개

2 2턴째

AI 응답을 넣고 새 질문을 추가합니다

```
messages = [  
  {  
    "role": "system",  
    "content": "Python 전문가"  
  },  
  {  
    "role": "user",  
    "content": "딕셔너리가 뭐야?"  
  },  
  {  
    "role": "assistant",  
    "content": "키-값 쌍 자료형..."  
  },  
  {  
    "role": "user",  
    "content": "리스트랑 뭐가 달라?"  
  }  
]
```

배열 크기: 4개

3 3턴째

배열이 계속 커집니다

```
messages = [  
  {  
    "role": "system",  
    "content": "Python 전문가"  
  },  
  {  
    "role": "user",  
    "content": "딕셔너리가 뭐야?"  
  },  
  {  
    "role": "assistant",  
    "content": "키-값 쌍 자료형..."  
  },  
  {  
    "role": "user",  
    "content": "리스트랑 뭐가 달라?"  
  },  
  {  
    "role": "assistant",  
    "content": "리스트는 순서가 있고..."  
  },  
  {  
    "role": "user",  
    "content": "예시 코드 보여줘"  
  }  
]
```

배열 크기: 6개

토큰 누적 문제

대화가 길어지면 비용이 증가하고 컨텍스트 윈도우 한계에 도달합니다

- ▶ 대화 1턴: 입력 약 100 토큰 — 비용 약 1원
- ▶ 대화 5턴: 입력 약 500 토큰 — 비용 약 5원
- ▶ 대화 20턴: 입력 약 2,000 토큰 — 비용 약 20원
- ▶ 대화 100턴: 입력 약 10,000 토큰 — 비용 약 100원, 점점 느려집니다
- ▶ 문제 1: 매 호출마다 이전 대화 전체를 보내므로 비용이 누적됩니다
- ▶ 문제 2: 모델의 최대 토큰 수(컨텍스트 윈도우) 한계를 넘을 수 있습니다

해결 방법	설명
대화 개수 제한	최근 N턴만 유지하고 오래된 대화를 삭제합니다
요약 후 교체	오래된 대화를 AI에게 요약시킨 뒤, 요약본으로 교체합니다
슬라이딩 윈도우	최근 메시지만 남기되 system 메시지는 항상 유지합니다

실습 가이드

5개 소스 파일을 순서대로 실행하며 핵심 개념을 관찰합니다

순서	소스 파일	내용	핵심 관찰 포인트
1	02/01.single-turn.py	단일 대화: 첫 API 호출	API 호출 한 번으로 질문하고 답변 받기
2	02/02.system_prompt.py	시스템 프롬프트: 역할 부여	같은 질문에 system 메시지가 답변을 바꾸는 것
3	03/01.multi-turn.py	대화 루프: 기본 멀티턴	while 루프로 계속 대화하는 구조
4	03/02.no_history.py	히스토리 없는 대화	같은 루프인데 맥락이 끊기는 현상 확인
5	03/03.with_history.py	히스토리 있는 대화	messages 배열에 이전 대화를 추가하여 맥락 유지

체크포인트

- ✓ model과 messages 파라미터의 역할을 설명할 수 있습니까?
- ✓ system, user, assistant 세 가지 role이 각각 어떤 역할을 하는지 설명할 수 있습니까?
- ✓ 히스토리 없이 대화하면 왜 맥락이 끊기는지 원인을 설명할 수 있습니까?
- ✓ 대화가 길어지면 토큰이 누적되는 문제가 왜 발생하는지 설명할 수 있습니까?

03

프롬프트 엔지니어링

- ✓ 프롬프트의 4가지 구성 요소(지시, 맥락, 입력, 출력 형식)를 구분합니다
- ✓ Zero-shot, One-shot, Few-shot의 차이를 예시로 설명할 수 있습니다
- ✓ 상황에 따라 적절한 프롬프트 기법을 선택할 수 있습니다

🕒 50분

프롬프트 엔지니어링이란

AI에게 원하는 결과를 얻기 위해 입력을 체계적으로 설계하는 기술입니다

프롬프트의 4가지 구성 요소

- 지시(Instruction): AI가 수행할 작업을 명시합니다
- 맥락(Context): 배경 정보, 역할, 제약 조건을 제공합니다
- 입력(Input): AI가 처리할 실제 데이터입니다
- 출력 형식(Output Format): 결과물의 구조와 형식을 지정합니다

나쁜 지시서 (모호한 프롬프트)

"리뷰 분석해 줘."

어떤 리뷰? 어떤 형식? 감성 분석? 주제 분석?

좋은 지시서 (구체적인 프롬프트)

"다음 고객 리뷰를 읽고, 감성(긍정/부정/중립), 주요 키워드, 개선 요청 사항을 JSON 형식으로 정리해 주십시오."

태스크, 입력, 출력 형식이 모두 명확합니다

시스템 프롬프트 작성법

시스템 프롬프트는 AI의 성격표입니다. 구체적일수록 답변 품질이 높아집니다.

항목	나쁜 예	좋은 예
역할 설정	"도우미입니다"	"3년차 백엔드 개발자, Python과 FastAPI 전문"
응답 형식	"잘 대답해주세요"	"3문장 이내, 코드 필요 시 Python 예시 포함"
제약 조건	(없음)	"모르는 내용은 모른다고 답하세요"
대상 설정	(없음)	"초등학생도 이해할 수 있는 쉬운 말로"
언어 설정	(없음)	"항상 한국어로 답변하세요"

작성 공식: 역할 + 대상 + 응답 형식 + 제약 조건

실전 예시

시스템 프롬프트 작성 예시

python

```
system_prompt = """
```

당신은 고객 상담 전문가입니다.

- 대상: 온라인 쇼핑몰 고객
- 말투: 존댓말, 친절하고 공감하는 어조
- 형식: 인사 한 줄, 답변 3문장 이내, 추가 질문 유도
- 제약: 환불/교환은 직접 처리하지 않고 담당자 연결을 안내하세요

```
"""
```

Zero-shot 프롬프팅

예시 없이 지시만으로 태스크를 수행합니다. "zero"는 예시 수가 0개라는 뜻입니다.

비유: 처음 만난 전문가에게 질문하기

상대방이 이미 충분한 지식을 갖고 있다고 가정하고, 원하는 것만 명확히 전달합니다. 별도의 예시나 샘플을 보여주지 않습니다.

관찰 포인트: 04/01.zero-shot.py

1. system 메시지에서 AI에게 어떤 역할을 부여하고 있는지 확인합니다 2. 지시와 입력 데이터가 어떻게 분리되어 있는지 살펴봅니다 3. 같은 프롬프트를 여러 번 실행했을 때 형식이 동일한지 확인합니다

```
zero-shot 예시 python

# Zero-shot: 예시 없이 지시만 전달
messages = [
    {"role": "system",
     "content": "고객 리뷰 분석 전문가입니다"},
    {"role": "user",
     "content": """다음 고객 리뷰의 감성을
긍정, 부정, 중립 중 하나로 분류하십시오.
감성만 한 단어로 답하십시오.

리뷰: "배송은 빨랐지만
포장 상태가 좋지 않았습니다."
"""}
]

# 기대 결과: "중립"
# 하지만 형식이 매번 달라질 수 있습니다
```

One-shot 프롬프팅

예시 1개를 제공하여 원하는 출력의 형식과 패턴을 보여줍니다

비유: 완성된 샘플 1개를 보여주기

"지난주 보고서를 참고해서 같은 형식으로 작성해 주십시오." 예시 하나만으로도 AI는 출력의 구조, 어투, 길이, 형식을 파악합니다.

[예시]

리뷰: "화면이 선명하고 배터리가 오래갑니다"

분석:

- 감성: 긍정
- 키워드: 화면, 배터리
- 개선 요청: 없음

[실제 입력]

리뷰: "배송은 빨랐지만 포장이..."

분석:

기준	설명	좋은 예
대표성	실제 처리할 데이터와 유사한 예시	같은 도메인의 리뷰
완전성	출력의 모든 필드를 포함	감성, 키워드, 개선 요청 모두 포함
명확성	모호하지 않은 결과	"감성: 긍정" (확실한 사례)
복잡도	실제 입력과 비슷한 난이도	긍정과 부정이 섞인 리뷰

Few-shot 프롬프팅

여러 예시를 제공하여 AI가 패턴을 더 정확히 학습하도록 유도합니다

예시 수	효과	토큰 사용량	적합한 상황
2개	기본 패턴 파악 가능	적음	단순 분류, 간단한 형식
3~5개	최적 구간, 패턴 정확 학습	보통	감성 분석, 카테고리 분류
5~10개	소폭 향상, 비용 증가	많음	복잡한 분류, 엡지 케이스
10개 이상	추가 효과 미미	매우 많음	대부분 불필요

- ▶ 다양성 확보: 가능한 모든 카테고리(긍정, 부정, 중립)를 예시에 포함합니다
- ▶ 경계 사례 포함: 판단이 어려운 사례를 하나 이상 포함합니다
- ▶ 일관된 형식: 모든 예시의 출력 형식을 동일하게 유지합니다
- ▶ 순서 고려: 예시 순서를 무작위로 배치하여 순서 의존을 방지합니다
- ▶ 3개에서 5개가 비용 대비 가장 효과적인 구간입니다

3가지 기법 종합 비교

태스크의 복잡도에 맞는 기법을 선택하는 것이 핵심입니다

비교 항목	Zero-shot	One-shot	Few-shot
예시 수	0개	1개	2~5개
토큰 비용	가장 적음	약간 증가	예시 수에 비례하여 증가
출력 형식 안정성	낮음 (매번 다를 수 있음)	보통 (예시 형식을 따름)	높음 (패턴 학습 효과)
정확도	단순 태스크에서 충분	형식이 중요할 때 효과적	복잡한 분류에서 우수
설정 난이도	가장 쉬움	좋은 예시 1개 선택 필요	다양한 예시 세트 구성 필요
적합한 상황	간단한 질의응답, 번역, 요약	특정 형식 출력, 보고서 양식	분류, 패턴 매칭, 구조화 분석
대표 사용 예	"이 문장을 영어로 번역하십시오"	"이 형식대로 보고서를 작성하십시오"	"이 기준으로 리뷰를 분류하십시오"

실전 프롬프트 작성 팁 7가지

Zero-shot, One-shot, Few-shot 어떤 기법을 사용하든 공통으로 적용됩니다

- ▶ 역할을 먼저 부여하십시오: 구체적인 전문가 역할을 부여하면 답변의 품질이 높아집니다
- ▶ 지시는 구체적이고 명확하게: '분석해 주십시오'보다 '긍정, 부정, 중립으로 분류하십시오'
- ▶ 출력 형식을 명시적으로 지정: JSON, CSV 등 구조화된 형식을 지정하면 파싱이 쉬워집니다
- ▶ 부정 지시보다 긍정 지시를 사용: '길게 쓰지 마십시오'보다 '3문장 이내로 작성하십시오'
- ▶ 구분자로 입력을 분리: 지시, 예시, 입력 데이터를 구분자(---, 삼중 따옴표)로 분리합니다
- ▶ 단계적 사고를 유도: '먼저 A를 판단하고, 그 다음 B를 판단하십시오' 형식이 정확도를 높입니다
- ▶ 반복 실험으로 개선: 결과 확인, 프롬프트 수정, 재실행을 반복하여 최적의 프롬프트를 찾습니다

체크포인트

- ✓ 프롬프트의 4가지 구성 요소(지시, 맥락, 입력, 출력 형식)를 각각 설명할 수 있습니까?
- ✓ Zero-shot, One-shot, Few-shot의 차이를 예시 수, 비용, 정확도 측면에서 비교할 수 있습니까?
- ✓ Few-shot이 항상 Zero-shot보다 좋지 않은 이유를 설명할 수 있습니까?
- ✓ '고객 리뷰를 5가지 카테고리로 분류하라'는 요청에 어떤 기법을 선택하시겠습니까?

04

Streamlit 웹 챗봇

- ✓ Streamlit이 무엇이고 왜 챗봇 UI에 적합한지 설명할 수 있습니다
- ✓ session_state의 동작 원리와 재실행 모델을 이해합니다
- ✓ 스트리밍 응답의 사용자 경험 효과를 이해합니다

🕒 25분

Streamlit이란

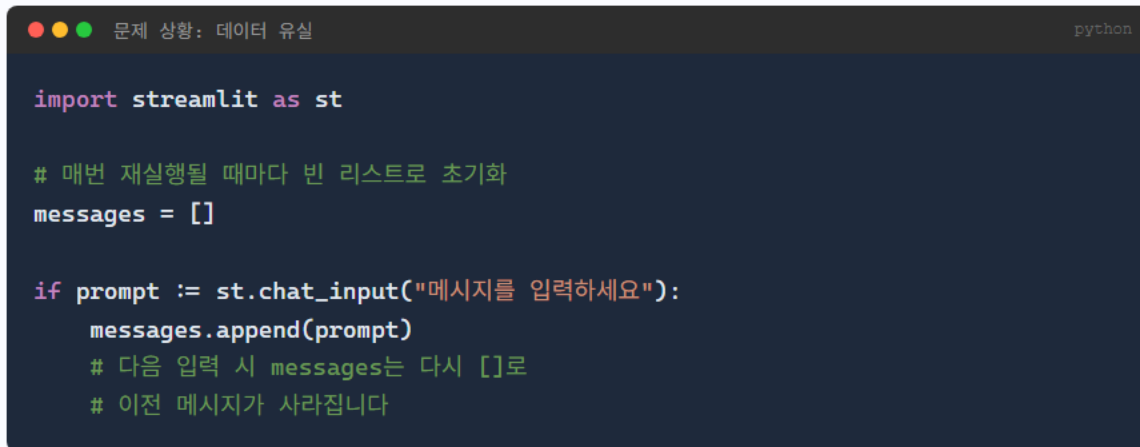
Python 코드만으로 웹 앱을 만드는 프레임워크입니다. HTML, CSS, JavaScript 없이 챗봇 UI를 구현할 수 있습니다.

비교 항목	Streamlit	Flask	Django
학습 난이도	매우 낮음 (Python만)	중간 (라우팅, 템플릿)	높음 (ORM, 미들웨어)
채팅 UI 구현	내장 컴포넌트 사용	HTML/JS 직접 작성	HTML/JS 직접 작성
스트리밍 지원	내장 함수 제공	WebSocket 직접 구현	WebSocket 직접 구현
상태 관리	session_state 내장	세션/DB 직접 관리	세션 프레임워크 제공
프로토타입 속도	30분 이내	2시간 이상	4시간 이상
HTML/CSS 필요	불필요	필수	필수

Streamlit의 재실행 모델

사용자 입력이 있을 때마다 Python 스크립트 전체를 처음부터 다시 실행합니다

- ▶ 사용자가 버튼 클릭 또는 입력을 제출합니다
- ▶ Streamlit이 Python 스크립트를 처음부터 끝까지 다시 실행합니다
- ▶ 화면이 새로 그려지며, 이전 화면은 완전히 사라집니다
- ▶ 일반 변수에 저장한 데이터는 매번 초기화됩니다



```
import streamlit as st

# 매번 재실행될 때마다 빈 리스트로 초기화
messages = []

if prompt := st.chat_input("메시지를 입력하세요"):
    messages.append(prompt)
    # 다음 입력 시 messages는 다시 []로
    # 이전 메시지가 사라집니다
```

session_state: 재실행에서 살아남는 저장소

일반 변수 = 칠판

수업이 끝나면(재실행되면) 지워집니다

session_state = 노트

수업이 끝나도(재실행되어도) 남아 있습니다

session_state 사용법

python

```
import streamlit as st

# session_state에 없으면 초기화
if "messages" not in st.session_state:
    st.session_state.messages = []

# 이전 메시지들을 화면에 표시
for msg in st.session_state.messages:
    with st.chat_message(msg["role"]):
        st.write(msg["content"])

# 새 메시지를 session_state에 저장
if prompt := st.chat_input("메시지를 입력하세요"):
    st.session_state.messages.append(
        {"role": "user", "content": prompt}
    )
```

핵심 채팅 컴포넌트

Streamlit이 제공하는 채팅 전용 UI 컴포넌트로 HTML/CSS 없이 깔끔한 채팅 화면을 만듭니다

```
chat_input / chat_message python

# chat_input: 화면 하단 고정 입력창
if prompt := st.chat_input("메시지를 입력하세요"):
    process_message(prompt)

# chat_message: 역할별 말풍선
with st.chat_message("user"):
    st.write("안녕하세요!")

with st.chat_message("assistant"):
    st.write("무엇을 도와드릴까요?")
```

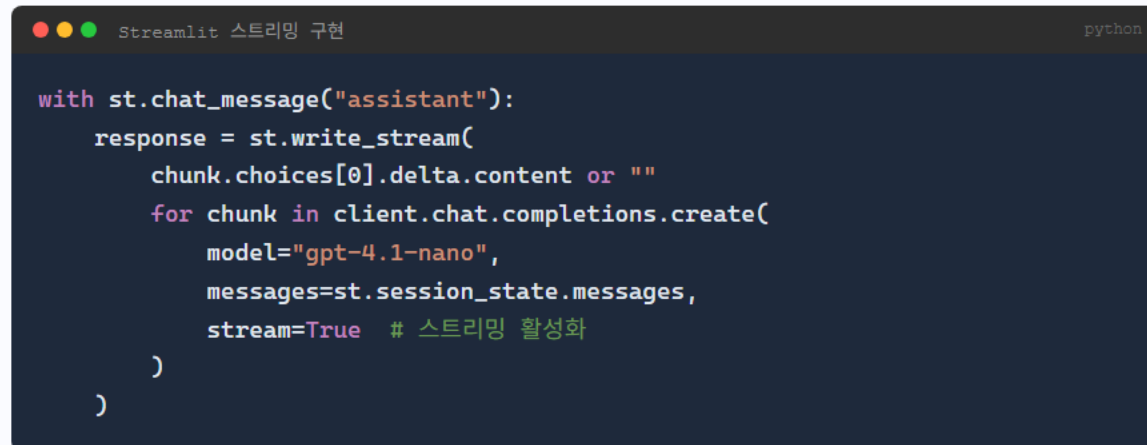
```
sidebar 설정 패널 python

# sidebar: 왼쪽 설정 패널
with st.sidebar:
    st.title("설정")
    model = st.selectbox(
        "모델",
        ["gpt-4.1-nano", "gpt-4.1-mini"]
    )
    temperature = st.slider(
        "Temperature", 0.0, 1.0, 0.7
    )
    system_prompt = st.text_area(
        "시스템 프롬프트",
        "당신은 친절한 AI입니다."
    )
```

스트리밍 응답

응답을 한 글자씩 출력하여 체감 대기 시간을 줄이는 기법입니다. 편지(일반)가 아닌 전화(스트리밍)와 같습니다.

항목	일반 응답	스트리밍 응답
첫 글자 표시	3~10초 (전체 생성 후)	0.5초 이내
대기 중 화면	빈 화면 / 로딩 스피너	글자가 타이핑되는 모습
체감 속도	느리게 느껴짐	빠르게 느껴짐
이탈률	높음 (기다리다 이탈)	낮음 (진행 상황 확인 가능)



```

with st.chat_message("assistant"):
    response = st.write_stream(
        chunk.choices[0].delta.content or ""
        for chunk in client.chat.completions.create(
            model="gpt-4.1-nano",
            messages=st.session_state.messages,
            stream=True # 스트리밍 활성화
        )
    )
  
```

실습 가이드

두 개의 파일을 순서대로 실행하며 Streamlit의 동작을 확인합니다

파일	내용	핵심 관찰 포인트
07/01.streamlit.py	Streamlit 기본 UI 구성	자동 브라우저 열림, localhost:8501 접속, 핫 리로드 확인
07/02.streamlit_chat.py	채팅 UI (사이드바, 스트리밍)	대화 유지 여부, 스트리밍 효과, 사이드바 설정 변경

체크포인트

- ✓ Streamlit이 Flask/Django와 어떻게 다른지, 왜 챗봇 프로토타입에 적합한지 설명할 수 있습니까?
- ✓ Streamlit의 재실행 모델이 무엇인지, 왜 session_state가 필요한지 설명할 수 있습니까?
- ✓ chat_input, chat_message, session_state 세 가지 컴포넌트의 역할을 각각 설명할 수 있습니까?
- ✓ 스트리밍 응답과 일반 응답의 사용자 경험 차이를 설명할 수 있습니까?

05

LangChain 입문

- ✓ LangChain이 무엇이고 왜 필요한지 설명할 수 있습니다
- ✓ OpenAI API 직접 호출과 LangChain 사용의 차이를 이해합니다
- ✓ LangSmith를 통한 트레이싱 개념과 실용적 가치를 이해합니다

🕒 25분

LangChain이 필요한 이유

OpenAI API만으로는 기능이 복잡해질수록 직접 구현할 것이 급격히 늘어납니다. 나무 깎기(직접 구현)가 아닌 레고 블록(LangChain) 조립 방식입니다.

문제	OpenAI API만 사용	LangChain 사용
프롬프트 관리	코드에 하드코딩	PromptTemplate으로 분리
모델 전환	코드 전체 재작성	모델 객체만 교체
체인 연결	함수 직접 연결	파이프 연산자로 선언적 연결
대화 히스토리	리스트 직접 관리	메모리 컴포넌트 제공
도구 연동	Function Calling 직접 구현	Tool/Agent 프레임워크 제공
디버깅	print문으로 확인	LangSmith로 시각적 추적
스트리밍	청크 파싱 직접 구현	.stream() 메서드 제공

LangChain 생태계

LangChain은 단일 라이브러리가 아니라 4개 제품으로 구성된 생태계입니다

LangChain 오늘 사용

LLM 앱 개발 프레임워크. 모델, 프롬프트, 체인, 도구, Retriever 등 핵심 컴포넌트 제공

LangGraph 심화

상태 기반 멀티 에이전트 프레임워크. 복잡한 워크플로우를 그래프로 설계

LangGraph Platform 프로덕션

에이전트를 프로덕션에 배포, 모니터링, 스케일링하는 클라우드 서비스

LangSmith 오늘 사용

LLM 호출 추적, 성능 모니터링, 평가, 프롬프트 관리. 모든 제품과 독립적으로 연동됩니다

LangSmith는 별도 제품으로, LangChain이나 LangGraph와 독립적으로 사용할 수 있습니다. 오늘 워크샵에서는 LLM 호출을 추적하는 용도로 활용합니다.

LangChain 핵심 컴포넌트

LangChain의 7가지 컴포넌트를 조합하여 LLM 애플리케이션을 구성합니다. 오늘 7가지를 모두 다룹니다.

컴포넌트	역할	예시	오늘 다루는 시점
Model	LLM과 통신하는 인터페이스	ChatOpenAI, Ollama	이번 LO
Prompt	동적 프롬프트 템플릿	ChatPromptTemplate	LCEL 체인 구성
Chain	컴포넌트를 연결하는 파이프라인	LCEL 파이프 연산자	LCEL 체인 구성
Retriever	외부 문서에서 관련 정보를 검색	VectorStoreRetriever	RAG 파이프라인
Tool	LLM이 호출하는 외부 기능	검색, 계산, API 호출	Tool과 Agent
Agent	도구를 자동 선택하고 실행	create_tool_calling_agent	Tool과 Agent
Memory	대화 히스토리 관리	메시지 히스토리 저장	이번 LO

LangSmith 트레이싱

LLM 호출의 입력, 출력, 소요 시간, 토큰 사용량을 실시간으로 추적합니다. 블랙박스에 유리 창문을 다는 것과 같습니다.

LangSmith 트레이스 상세

Run: ChatOpenAI (gpt-4.1-nano)

Status: 성공

지연 시간: 1.2초

토큰: 입력 45 / 출력 128 / 합계 173

비용: \$0.0002

[입력]

System: 당신은 친절한 AI입니다.

Human: 파이썬이 뭐야?

확인 가능 항목

활용 방법

입력 메시지

프롬프트가 의도대로 전달되었는지 확인

출력 메시지

응답 품질을 평가

지연 시간

느린 호출을 식별하고 최적화

토큰 사용량

비용 모니터링 및 프롬프트 최적화

에러 로그

실패 원인을 신속하게 파악

체인 실행 순서

복잡한 체인에서 각 단계 추적

메시지 히스토리 관리

OpenAI 방식(딕셔너리 리스트)과 LangChain 방식(Message 객체)을 비교합니다

항목	OpenAI 방식	LangChain 방식
메시지 형식	딕셔너리 {"role": ..., "content": ...}	Message 객체 (HumanMessage, AIMessage)
타입 안정성	오타 위험 (문자열 기반)	타입 체크 가능 (객체 기반)
응답 추가	수동 딕셔너리 생성	응답 객체 직접 추가
체인 호환	불가	LCEL 체인에 바로 연결 가능
확장성	별도 구현 필요	메모리 컴포넌트와 연동 가능

실습 가이드

세 개의 파일을 순서대로 실행하며 LangChain의 기본 사용법을 확인합니다

파일	내용	핵심 관찰 포인트
08/01.langchain.py	LangChain 첫 호출	ChatOpenAI 객체 생성, .invoke() 호출, response.content 확인
08/02.langsmith_example.py	LangSmith 트레이싱 확인	환경 변수 설정, 대시보드에서 트레이스 확인, 토큰/비용 조회
08/03.message_history.py	메시지 히스토리 관리	Message 객체 사용, 히스토리 누적, 맥락 유지 확인

체크포인트

- ✓ LangChain을 쓰면 OpenAI API 직접 호출 대비 어떤 이점이 있는지 세 가지 이상 설명할 수 있습니까?
- ✓ ChatOpenAI와 OpenAI 클라이언트의 코드 구조 차이를 이해했습니까?
- ✓ LangSmith 트레이싱이 무엇이고, 왜 디버깅에 유용한지 설명할 수 있습니까?
- ✓ LangChain의 5가지 핵심 컴포넌트(모델, 프롬프트, 체인, 메모리, 도구)를 열거할 수 있습니까?

06

LCEL 체인 구성

- ✓ LCEL의 파이프 연산자로 체인을 구성하는 방법을 이해합니다
- ✓ 출력 파서와 프롬프트 템플릿을 활용할 수 있습니다
- ✓ 단순 체인에서 복합 체인까지 확장하는 과정을 이해합니다

🕒 50분

LCEL이란

LangChain Expression Language: 컴포넌트를 파이프 연산자로 연결하여 체인을 구성하는 문법입니다

레고 블록 비유

레고 조립: [머리] + [몸통] + [다리] = 완성된 인형

LCEL 체인: [프롬프트] | [LLM] | [파서] = 완성된 파이프라인

```
기본 LCEL 체인

from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# 1. 프롬프트 템플릿
prompt = ChatPromptTemplate.from_template(
    "{topic}에 대해 초보자도 이해할 수 있게 설명해주세요."
)

# 2. LLM
llm = ChatOpenAI(model="gpt-4.1-nano")

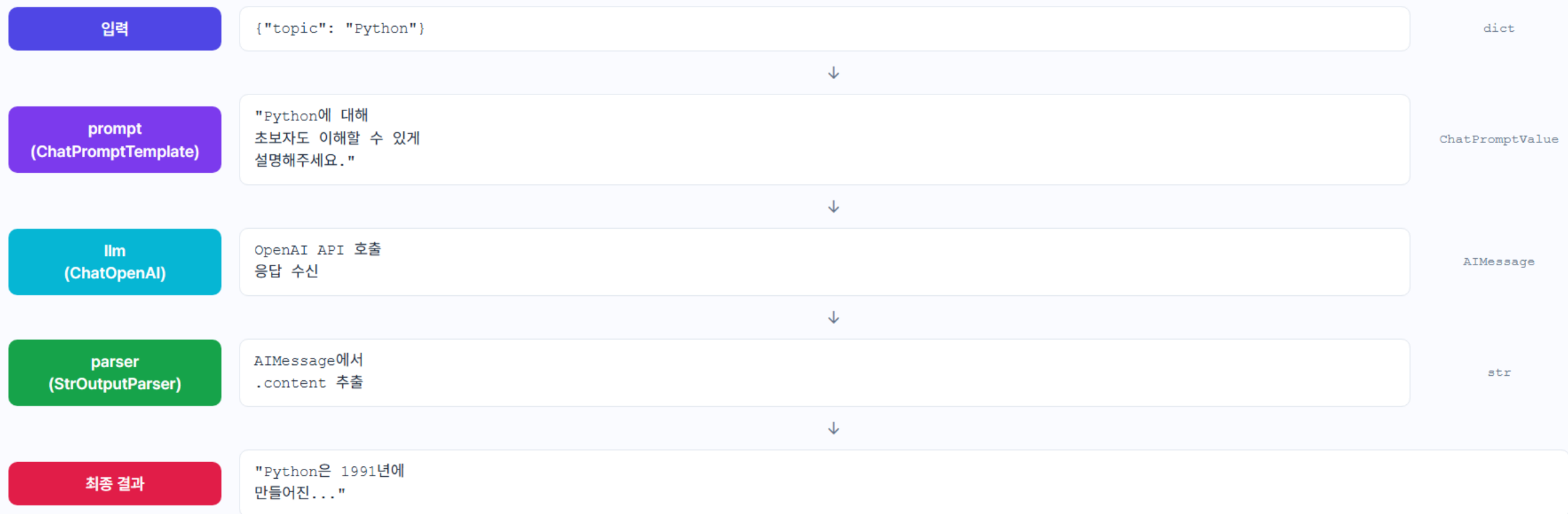
# 3. 출력 파서
parser = StrOutputParser()

# 파이프 연산자로 연결
chain = prompt | llm | parser

result = chain.invoke({"topic": "Python"})
```

파이프 연산자 동작 원리

왼쪽 컴포넌트의 출력이 오른쪽 컴포넌트의 입력으로 전달됩니다: chain = prompt | llm | parser



OutputParser 4종 비교

LLM의 자유 형식 출력을 프로그램이 이해하는 구조화된 데이터로 변환하는 통역사입니다

파서	용도	출력 타입	사용 예시
StrOutputParser	AIMessage에서 텍스트만 추출	str	일반 대화, 설명 생성
JsonOutputParser	JSON 형식으로 파싱	dict	API 응답, 구조화 데이터
CommaSeparated ListOutputParser	콤마로 구분된 리스트로 파싱	list	태그 추출, 키워드 목록
PydanticOutputParser	Pydantic 모델로 파싱 (스키마 검증)	Pydantic 객체	엄격한 타입이 필요한 경우

PromptTemplate vs ChatPromptTemplate

워크샵에서는 챗봇을 만들고 있으므로 역할 구분이 가능한 ChatPromptTemplate을 사용합니다

항목	PromptTemplate	ChatPromptTemplate
용도	단순 텍스트 생성	대화형 AI (챗봇)
역할 구분	없음	system, human, ai 구분
ChatOpenAI 호환	변환 필요	직접 사용 가능
권장 사용처	텍스트 완성 작업	챗봇, 대화형 체인

```

ChatPromptTemplate 사용 예시

# ChatPromptTemplate: 역할 기반 메시지 템플릿
template = ChatPromptTemplate.from_messages([
    ("system", "당신은 {role}입니다. {style}로 답변합니다."),
    ("human", "{question}")
])
result = template.invoke({"role": "Python 튜터", "style": "초보자에게 친절하게", "question": "리스트가 뭐야?"})
  
```

체인은 이렇게 확장됩니다

단순한 연결에서 시작하여 다단계 파이프라인까지, LCEL은 같은 문법으로 확장됩니다

1 질문 하나, 답변 하나

사용자가 물어보면 AI가 답한다

```
prompt | llm | parser
```

프롬프트, 모델, 파서 3개를 연결하는 가장 기본적인 형태입니다

2 상황에 따라 다르게 질문

같은 체인에 다른 변수를 넣어 재사용한다

```
chain.invoke({"topic": "Python"})  
chain.invoke({"topic": "Java"})
```

템플릿 변수만 바꾸면 동일한 체인을 여러 용도로 활용할 수 있습니다

3 AI 결과를 다음 AI에 전달

키워드 추출 후 그 키워드로 설명을 생성한다

```
{"keywords": 추출_체인} | 설명_체인
```

첫 번째 체인의 출력이 두 번째 체인의 입력이 됩니다. 다단계 처리의 핵심입니다

4 한 글자씩 실시간 출력

같은 체인을 스트리밍으로 전환한다

```
chain.stream() # 한 글자씩  
chain.batch() # 여러 건 동시
```

체인을 바꾸지 않고 실행 방식만 바꾸면 스트리밍과 일괄 처리가 됩니다

스트리밍 체인

LCEL로 구성한 체인은 `.invoke()`, `.stream()`, `.batch()` 메서드를 모두 자동으로 지원합니다

```
.invoke() vs .stream() vs .batch() python

chain = prompt | llm | parser

# 동기 실행: 전체 결과를 한 번에 반환
result = chain.invoke({"topic": "Python"})

# 스트리밍: 토큰 단위로 순차 반환
for chunk in chain.stream({"topic": "Python"}):
    print(chunk, end="", flush=True)

# 일괄 처리: 여러 입력을 동시에 실행
results = chain.batch([
    {"topic": "Python"},
    {"topic": "JavaScript"},
    {"topic": "Rust"},
])
```

메서드	용도	반환 방식
<code>.invoke()</code>	단일 실행	전체 결과를 한 번에 반환
<code>.stream()</code>	스트리밍	토큰 단위로 순차 반환
<code>.batch()</code>	일괄 처리	여러 입력을 병렬로 처리

실습 가이드

7개 파일을 순서대로 실행하며 LCEL 체인 구성을 익힙니다

파일	내용	핵심 관찰 포인트
08/04.langchain_vs_openai.py	OpenAI vs LangChain 비교	코드 길이 차이, 응답 접근 방식 비교
08/05.langchain_streaming.py	LangChain 스트리밍	.stream() 호출, 청크 단위 출력 확인
09/01.lcel_basic.py	LCEL 기본 체인	파이프 연산자, invoke 입력, 출력 타입 확인
09/02.lcel_parser.py	출력 파서	StrOutputParser와 JsonOutputParser 비교
09/03.prompt_template.py	프롬프트 템플릿	변수 치환, 역할 구분, 재사용성 확인
09/04.prompt_interactive.py	대화형 템플릿	MessagesPlaceholder, 맥락 유지 확인
09/05.lcel_complex_chain.py	복잡한 체인 연결	다단계 처리, 중간 결과, LangSmith 트레이스

체크포인트

- ✓ LCEL의 파이프 연산자가 데이터를 어떻게 전달하는지 설명할 수 있습니까?
- ✓ StrOutputParser와 JsonOutputParser의 차이와 용도를 설명할 수 있습니까?
- ✓ ChatPromptTemplate에서 변수 치환이 어떻게 작동하는지 이해했습니까?
- ✓ LCEL 체인이 .invoke(), .stream(), .batch()를 모두 지원한다는 것을 이해했습니까?

07

LangChain Tool과 Agent

- ✓ Tool의 개념과 LLM에 도구가 필요한 이유를 이해합니다
- ✓ Agent가 도구를 자동으로 선택하고 실행하는 과정을 이해합니다
- ✓ 외부 API(Wikipedia 등)를 Agent에 연동하는 방법을 이해합니다

🕒 50분

Tool이 필요한 이유

LLM은 매우 똑똑하지만, 혼자서는 할 수 없는 일이 있습니다. Tool은 AI에게 망치, 드라이버 같은 도구를 주는 것입니다.

LLM이 혼자 할 수 없는 일	이유
오늘 날씨 확인	학습 데이터 이후의 실시간 정보를 모릅니다
정확한 계산 (738 x 9247)	답을 추측하므로 틀릴 수 있습니다
이메일 전송	외부 시스템에 접근하는 능력이 없습니다
데이터베이스 조회	내부 데이터에 접근할 수 없습니다
최신 뉴스 검색	학습 데이터의 시점 이후 정보를 모릅니다

도구 없는 AI

"738 x 9247은... 대략 6,800,000쯤?"

도구 있는 AI

[계산기 호출] 결과: 6,824,286

Tool 구조

LangChain에서 Tool은 이름, 설명, 함수 세 가지 요소로 구성됩니다. 설명(docstring)이 도구 선택의 핵심입니다.

```
@tool 데코레이터  
  
from langchain_core.tools import tool  
  
@tool  
def calculator(expression: str) → str:  
    """수학 계산을 수행합니다.  
    수식을 문자열로 받아 결과를 반환합니다.  
    """  
    return str(eval(expression))
```

요소	역할	예시
이름	도구를 식별하는 이름	calculator
설명 (docstring)	LLM이 도구를 언제 사용할지 판단하는 기준	수학 계산을 수행합니다
함수	실제 실행되는 코드	eval(expression)

Tool Calling 4단계

LLM은 도구를 직접 실행하지 않습니다. 어떤 도구를 호출할지 지시만 하고, 실제 실행은 시스템 코드가 합니다.

1단계

사용자: 질문을 합니다

"서울 날씨 어때?"

2단계

LLM: 질문을 분석하고 도구를 결정합니다

```
{"tool": "get_weather", "args": {"city": "서울"}}
```

3단계

시스템 (코드): LLM의 지시대로 도구를 실제 실행합니다

```
get_weather("서울") --> "서울: 맑음, 15도"
```

4단계

LLM: 도구 결과로 최종 답변을 생성합니다

"서울의 현재 날씨는 맑음이며, 기온은 15도입니다."

Agent의 사고 과정

Agent는 생각하고, 도구를 쓰고, 결과를 보고, 다시 생각하는 과정을 반복합니다

 **사용자 질문** "아인슈타인이 태어난 해와 사망한 해를 알려줘"

 **생각** "Wikipedia에서 찾아봐야겠다"

 **도구 호출** wikipedia_search("Albert Einstein")

 **결과 확인** "1879년 3월 14일 출생, 1955년 4월 18일 사망..."

 **다시 생각** "필요한 정보를 얻었다. 답변할 수 있다."

 **최종 답변** "1879년에 태어나서 1955년에 사망했습니다"

verbose=True 출력 (실제 터미널)

```
> Entering new AgentExecutor chain...
```

```
Thought: Wikipedia에서 검색해야겠다
```

```
Invoking: `wikipedia_search`  
with {"query": "Albert Einstein"}
```

```
Albert Einstein (14 March 1879 -  
18 April 1955) was a German-born...
```

```
Thought: 정보를 충분히 얻었다
```

```
Final Answer: 아인슈타인은 1879년에  
태어나서 1955년에 사망했습니다.
```

```
> Finished chain.
```

verbose=True로 실행하면 Agent의 사고 과정을 터미널에서 직접 관찰할 수 있습니다

Agent 라우팅

Agent는 질문을 분석하여 여러 도구 중 가장 적절한 것을 선택합니다. 도구의 설명(description)이 선택의 핵심 기준입니다.

사용자 질문	Agent 라우팅 결과	선택 근거
"15의 제곱근은?"	calculator 선택	"수학 계산을 수행합니다" 매칭
"아인슈타인은 누구?"	wikipedia 선택	"Wikipedia에서 정보를 검색합니다" 매칭
"오늘 서울 날씨?"	weather 선택	"현재 날씨를 조회합니다" 매칭
"파이썬이 뭐야?"	도구 없이 직접 답변	일반 지식이므로 도구가 불필요
"아인슈타인이 태어난 해의 제곱은?"	wikipedia + calculator	복합 질문: 두 도구를 순차 호출

Wikipedia Agent

LLM의 학습 데이터에 없는 정보를 실시간으로 검색하여 정확한 답변을 생성합니다

상황	LLM 단독	Wikipedia Agent
최신 정보	학습 이후 정보 없음	Wikipedia의 최신 문서 검색
정확한 수치	근사값을 추측	정확한 데이터 인용
출처 제공	출처 없음	Wikipedia 문서 링크 제공
환각 방지	없는 정보를 만들어낼 수 있음	실제 문서 기반 답변

이것이 RAG의 시작입니다

Wikipedia Agent: 외부 지식(Wikipedia) 검색 후 답변 생성 | RAG: 외부 지식(사내 문서, PDF 등) 검색 후 답변 생성. 워크샵 후반부에서 배울 RAG는 이 개념을 사내 문서로 확장한 것입니다.

실습 가이드

네 개의 파일을 순서대로 실행하며 Tool과 Agent의 동작을 확인합니다

파일	내용	핵심 관찰 포인트
10/01.tool_basic.py	Tool 정의 기본	@tool 데코레이터, docstring 역할, 타입 힌트 확인
10/02.tool_agent.py	Agent 자동 실행	자동 도구 선택, 도구 실행, verbose 출력 확인
10/03.agent_routing.py	Agent 라우팅 관찰	질문별 다른 도구 선택, 직접 답변, 다중 도구 호출
10/04.tool_wikipedia.py	Wikipedia 검색 Agent	외부 지식 검색, 정확도 비교, RAG와의 연관성

체크포인트

- ✓ LLM에 Tool이 필요한 이유를 세 가지 이상 설명할 수 있습니까?
- ✓ Tool Calling에서 LLM이 도구를 직접 실행하는 것이 아니라 호출 요청만 한다는 것을 이해했습니까?
- ✓ Agent의 Thought, Action, Observation 루프를 설명할 수 있습니까?
- ✓ Wikipedia Agent가 RAG의 기초 형태라는 연결 고리를 이해했습니까?

08

고객상담 챗봇과 RAG 필요성

- ✓ 시스템 프롬프트에 FAQ를 넣어 고객상담 챗봇을 만드는 원리를 이해합니다
- ✓ FAQ 규모별 토큰 수, 비용, 정확도 변화를 체감합니다
- ✓ 시스템 프롬프트 방식의 한계와 RAG 전환 동기를 설명할 수 있습니다

🕒 25분

고객상담 챗봇, 어디서든 만납니다

쇼핑몰, 은행, 통신사, 배달 앱까지. AI 챗봇은 이미 고객 상담의 핵심 채널입니다.



쇼핑몰

배송 조회, 반품 안내, 교환 절차를 24시간 자동 응대합니다



금융

계좌 조회, 카드 분실 신고, 대출 상담을 즉시 처리합니다



통신사

요금제 변경, 데이터 잔여량, 해지 안내를 자동으로 안내합니다



배달/서비스

주문 상태, 예상 도착 시간, 환불 요청을 실시간 처리합니다

이번 시간 목표 가장 간단한 방법으로 고객상담 챗봇을 만들어 보고, 그 한계를 직접 체험한 뒤, 이를 해결하는 RAG를 배웁니다

시스템 프롬프트 방식의 고객상담 챗봇

FAQ 전체를 시스템 메시지에 넣어 LLM이 참고하도록 하는 가장 단순한 접근법입니다

```
cs_chatbot.py python

from langchain_openai import ChatOpenAI
from langchain_core.messages import SystemMessage, HumanMessage

faq_data = """
Q: 배송은 얼마나 걸리나요?
A: 주문 후 2-3영업일 이내 배송됩니다.

Q: 반품은 어떻게 하나요?
A: 수령 후 7일 이내 고객센터로 연락하시면 됩니다.
"""

system_prompt = f"""당신은 고객상담 챗봇입니다.
아래 FAQ를 참고하여 친절하게 답변하세요.
=== FAQ 목록 ===
{faq_data}"""

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
response = llm.invoke([
    SystemMessage(content=system_prompt),
    HumanMessage(content="배송 얼마나 걸려요?")
])
```

- ✓ 구현이 단순합니다 (코드 10줄 이내)
- ✓ 별도 데이터베이스가 필요 없습니다
- ✓ FAQ 수정이 즉시 반영됩니다
- ✓ 소규모 서비스에는 충분합니다

FAQ 규모별 비교: 토큰, 비용, 정확도

FAQ가 늘어나면 비용은 선형 증가하고, 정확도는 급격히 저하됩니다

항목	FAQ 10개	FAQ 100개	FAQ 1,000개
FAQ 토큰 수	약 700	약 7,000	약 70,000
GPT-4o-mini 1회 비용	\$0.00014	\$0.0011	\$0.011
하루 1,000건 비용	\$0.14	\$1.1	\$11
컨텍스트 윈도우 점유율	0.7%	5.6%	54.7%
응답 정확도	매우 높음	높음	낮음
응답 속도	빠름	보통	느림

한계 체험 시나리오: FAQ가 많아지면

cs_limit.py 실습에서 직접 체험하게 될 4가지 현상입니다

- 정보 혼동(Confusion): 비슷한 주제의 FAQ가 여러 개 있으면 LLM이 뒤섞인 답변을 생성합니다. 일반 배송 기간과 해외 배송 기간을 혼동하여 틀린 조합이 나타납니다.
- 맥락 상실(Lost in the Middle): 시스템 프롬프트 중간에 위치한 FAQ는 앞뒤에 비해 참조되지 않을 확률이 높습니다. FAQ #50번에 대한 질문을 해도 엉뚱한 답이 나올 수 있습니다.
- 컨텍스트 윈도우 초과: FAQ 1,000개(약 70,000 토큰)에 대화 이력까지 더하면 128K 토큰 한계에 근접합니다. 대화가 길어지면 초기 FAQ 정보가 잘려나갑니다.
- 업데이트의 악몽: FAQ를 시스템 프롬프트에 하드코딩하면, 상품 가격 변경이나 시즌 이벤트 FAQ 추가 시 매번 코드를 수정하고 재배포해야 합니다.

시스템 프롬프트 방식 vs RAG 방식

핵심 차이: 모든 정보를 매번 통째로 보내는 방식에서, 질문에 관련된 정보만 검색하여 보내는 방식으로 전환합니다

비교 항목	시스템 프롬프트 방식	RAG 방식
데이터 전달	FAQ 전체 1,000개 전송	관련 FAQ 3개만 검색 후 전송
토큰 소비	매번 약 70,000 토큰	매번 약 500 토큰
비용 비율	1배 (기준)	약 1/140
정확도	FAQ 많을수록 저하	관련 문서만 참조하여 높음
업데이트	코드 수정 후 재배포	DB에 추가만 하면 완료
확장성	컨텍스트 윈도우 한계	문서 수 제한 없음

실습 가이드

두 개의 실습 파일로 시스템 프롬프트 방식의 장점과 한계를 직접 체험합니다

실습 파일	목적	핵심 관찰 포인트
cs_chatbot.py	시스템 프롬프트 방식 챗봇 구현	FAQ에 있는 질문에 정확히 답변하는지 확인합니다
cs_chatbot.py	FAQ 외 질문 처리 확인	FAQ에 없는 질문에 고객센터 안내가 나오는지 확인합니다
cs_chatbot.py	토큰 사용량 측정	입력/출력 토큰 수를 기록합니다
cs_limit.py	FAQ 규모별 한계 체험	FAQ 10개, 100개, 500개에서 토큰 수 변화를 추적합니다
cs_limit.py	응답 품질 저하 관찰	동일 질문의 답변 품질이 FAQ 수에 따라 변하는지 비교합니다
cs_limit.py	Lost in the Middle 체험	중간 위치 FAQ에 대한 질문으로 맥락 상실 현상을 확인합니다

체크포인트

- ✓ 시스템 프롬프트 방식의 고객상담 챗봇이 어떤 구조로 작동하는지 설명할 수 있습니까?
- ✓ FAQ 10개일 때와 1,000개일 때의 토큰 수, 비용, 정확도 차이를 말할 수 있습니까?
- ✓ Lost in the Middle 현상이 무엇이고 왜 발생하는지 설명할 수 있습니까?
- ✓ RAG가 '전체 전송' 대신 '검색 후 전송'을 하는 이유를 비용과 정확도 관점에서 설명할 수 있습니까?

09

RAG 이론

- ✓ LLM의 3가지 근본 한계(지식 컷오프, 할루시네이션, 도메인 무지)를 설명할 수 있습니다
- ✓ RAG의 3단계(인덱싱, 검색, 생성)를 상세히 이해합니다
- ✓ 임베딩, 청킹, 벡터 데이터베이스의 원리를 이해합니다

🕒 25분

LLM의 3가지 근본 한계

RAG는 이 세 가지 한계를 한 번에 해결합니다



지식 컷오프

LLM은 학습 시점까지의 데이터만 알고 있습니다. 어제 발표된 신제품, 오늘 변경된 정책은 답할 수 없습니다.



할루시네이션

모르는 것도 자신 있게 답변합니다. 존재하지 않는 논문을 인용하거나, 없는 제품 스펙을 만들어냅니다.



도메인 무지

범용 지식은 풍부하지만 특정 조직의 내부 지식(사내 규정, 매뉴얼, 계약 조건)은 알 수 없습니다.

RAG 3단계

인덱싱(준비), 검색(찾기), 생성(답변). 도서관 사서의 업무 과정과 같습니다.

1단계

인덱싱

사전 준비 (1회)

문서 로딩 , 청크 분할 , 임베딩 변환 , 벡터DB 저장

새 책이 도서관에 들어오면 목차별로 나누고, 키워드 색인을 만들어 서가에 배치합니다

2단계

검색

질문할 때마다

질문 임베딩 , 벡터DB 유사도 검색 , 관련 문서 Top-K 반환

사서에게 질문하면 색인 카드를 뒤져서 관련 있는 책 페이지 몇 군데를 찾아줍니다

3단계

생성

검색 직후

검색 문서 + 질문 , LLM 전달 , 근거 기반 답변

사서가 찾아준 페이지를 읽고, 그 내용을 바탕으로 질문자에게 설명해줍니다

임베딩: "의미의 좌표"

텍스트의 의미를 1536차원 숫자 벡터로 변환합니다. 의미가 비슷하면 벡터도 가깝습니다.

```
# 임베딩 벡터 예시 (1536차원)
```

```
"강아지" [0.82, -0.15, 0.43, 0.67, ...]
```

```
"개" [0.79, -0.12, 0.41, 0.70, ...] (매우 가까움)
```

```
"고양이" [0.65, -0.08, 0.38, 0.55, ...] (비교적 가까움)
```

```
"자동차" [-0.23, 0.91, -0.17, 0.05, ...] (매우 멀음)
```

코사인 유사도 검색

반품 및 환불 절차 안내

0.92

교환 신청 방법

0.78

포인트 적립 안내

0.23

1.0 = 완전히 같은 의미 / 0.0 = 전혀 관련 없음

청킹: 문서를 적절한 크기로 나누기

chunk_size와 chunk_overlap 설정에 따라 검색 정확도와 문맥 보존이 달라집니다

항목	작은 청크 (200자)	중간 청크 (500자)	큰 청크 (2000자)
검색 정밀도	높음	중간	낮음
문맥 보존	낮음	중간	높음
벡터 수	많음	중간	적음
권장 용도	FAQ, QnA	일반 문서	논문, 법률 문서

파라미터	역할	작게 설정하면	크게 설정하면
chunk_size	청크 최대 크기	검색 정밀도 높음, 맥락 부족	맥락 풍부, 검색 정밀도 낮음
chunk_overlap	인접 청크 겹침	경계 정보 손실 가능	경계 정보 보존, 저장 공간 증가

벡터 데이터베이스: 의미 기반 검색

핵심 차이: "키워드 일치"가 아닌 "의미 일치"로 검색합니다. "돈 돌려받고 싶어요"로 "환불 절차" 문서를 찾을 수 있습니다.

항목	관계형 DB (MySQL 등)	벡터 DB (ChromaDB 등)
저장 형태	행과 열 (테이블)	벡터와 메타데이터
검색 방식	정확한 값 일치 (WHERE 조건)	유사도 기반 근접 검색
검색어 "환불 방법"	"환불"이 포함된 행만 반환	"반품", "교환", "환불 절차" 등 의미적으로 유사한 결과 반환
인덱스	B-Tree, Hash	HNSW, IVF 등 벡터 전용 인덱스
장점	정확한 조건 검색, 트랜잭션	의미 기반 검색, 유연한 질의
대표 제품	MySQL, PostgreSQL	ChromaDB, Pinecone, Weaviate

RAG 전체 아키텍처

인덱싱(사전 준비)과 검색 및 생성(질문 시), 두 파이프라인이 벡터DB로 연결됩니다

1단계 인덱싱 (사전 준비, 1회 실행)

원본 문서
PDF, TXT

›

문서 로딩
Document Loader

›

청킹
500자 단위 분할

›

임베딩
벡터 변환

›

벡터DB 저장
ChromaDB

2~3단계 검색과 생성 (질문할 때마다 실행)

사용자 질문

›

질문 임베딩
벡터 변환

›

유사도 검색
Top-K 반환

›

프롬프트 조립
질문 + 문서

›

근거 기반 답변
LLM 생성

1단계의 벡터DB가 2단계 유사도 검색의 대상이 됩니다. 두 파이프라인은 벡터DB로 연결됩니다.

RAG 기업 활용 사례

RAG는 이미 다양한 산업에서 실전 적용되고 있습니다



고객 서비스 자동화

이커머스, 통신사, 금융사의 고객상담 챗봇에 적용합니다. FAQ, 상품 매뉴얼, 이용약관을 RAG로 연결하여 상담사 업무량을 40-60% 감소시킵니다.



사내 지식 검색

Confluence, Notion, SharePoint 문서를 RAG로 연결합니다. 신입사원 온보딩 시간을 단축하고 반복 질문을 해소합니다.



법률 및 규정 분석

법령, 판례, 계약서, 규정집을 RAG로 검색합니다. 관련 판례 검색 시간을 수 시간에서 수 분으로 단축합니다.



의료 정보 지원

의학 논문, 약품 정보, 진료 가이드라인을 RAG로 연결합니다. 최신 연구 결과에 기반한 의사결정을 지원합니다.

체크포인트

- ✓ LLM의 3가지 근본 한계(지식 컷오프, 할루시네이션, 도메인 무지)를 각각 설명할 수 있습니까?
- ✓ RAG의 3단계(인덱싱, 검색, 생성)를 순서대로 설명하고 각 단계에서 일어나는 일을 말할 수 있습니까?
- ✓ 임베딩을 '의미의 좌표'로 비유하여 비전공자에게 설명할 수 있습니까?
- ✓ chunk_size를 너무 작게 또는 너무 크게 설정하면 각각 어떤 문제가 생기는지 설명할 수 있습니까?
- ✓ 벡터 데이터베이스가 일반 관계형 데이터베이스와 어떻게 다른지 한 문장으로 말할 수 있습니까?

10

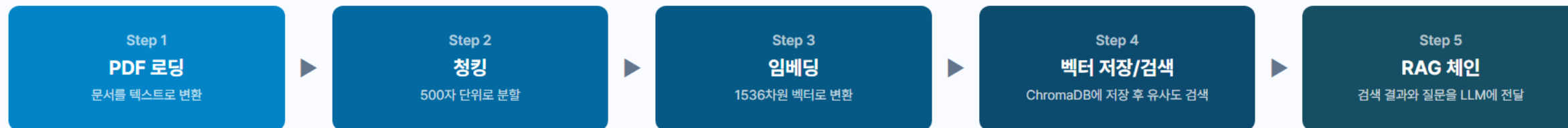
RAG 파이프라인 구현

- ✓ PDF 로딩부터 RAG 체인까지 5단계 파이프라인을 구현할 수 있습니다
- ✓ 청킹, 임베딩, 벡터 저장의 코드 구조를 이해합니다
- ✓ RAG 체인이 검색 결과를 LLM에 전달하는 흐름을 파악합니다

🕒 25분

RAG 파이프라인 전체 흐름

5단계를 순서대로 구현하면 완성형 RAG 시스템이 됩니다



준비 단계(Step 1-4)는 서비스 시작 전 한 번 실행합니다. 질문 응답 단계(Step 4-5)는 사용자가 질문할 때마다 실행됩니다.

PDF를 텍스트로 변환하기

PyPDFLoader가 PDF를 페이지별 Document 객체로 변환합니다

```
step1_load_pdf.py python

loader = PyPDFLoader("회사소개서.pdf")
documents = loader.load()

# 결과: 페이지별 Document 객체 리스트
# documents[0].page_content → 텍스트 내용
# documents[0].metadata → {"source": "회사소개서.pdf", "page": 0}
```

핵심

PDF의 각 페이지가 하나의 Document 객체가 됩니다. page_content에 텍스트, metadata에 출처 정보가 담깁니다.

주의

이미지 기반 PDF(스캔본)는 텍스트 추출이 안 됩니다. 텍스트가 있는 PDF를 사용해야 합니다.

문서를 검색 가능한 크기로 분할하기

500자 단위로 나누되, 50자를 겹쳐서 문맥이 끊기지 않게 합니다

```
step2_chunking.py python

splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,      # 청크 최대 글자 수
    chunk_overlap=50     # 인접 청크 겹침 (문맥 보존)
)

chunks = splitter.split_documents(documents)
# 5페이지 PDF → 약 20~30개 청크로 분할됩니다
```

왜 나누는가

페이지 전체를 벡터로 만들면 의미가 희석됩니다. 500자로 나누면 "반쯤 절차"만 담긴 청크가 생겨서 검색이 정확해집니다.

overlap이란

인접 청크가 50자 겹칩니다. 문장이 청크 경계에서 잘리더라도 다음 청크에 앞부분이 포함되어 문맥이 유지됩니다.

텍스트를 숫자 벡터로 변환하기

임베딩 모델이 텍스트의 의미를 1536개 숫자로 표현합니다

```
step3_embedding.py python

embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

vector = embeddings.embed_query("반품 절차가 어떻게 되나요?")
# 결과: [-0.023, 0.015, -0.008, 0.042, ...] (1536개 숫자)

# "환불 방법 알려주세요" → 비슷한 벡터 (의미가 가까우므로)
# "오늘 날씨 어때?" → 완전히 다른 벡터 (의미가 다르므로)
```

핵심

의미가 비슷한 텍스트는 비슷한 벡터가 됩니다. 이 성질 덕분에 "환불"을 검색해도 "반품 절차" 문서가 찾아집니다. GPS 좌표처럼 의미의 위치를 숫자로 나타낸 것입니다.

벡터를 저장하고 유사한 문서 검색하기

ChromaDB에 저장한 뒤, 질문과 가장 가까운 청크 3개를 찾아옵니다

```
step4_vector_store.py python

# 저장: 청크를 자동으로 임베딩하여 ChromaDB에 저장
vectorstore = Chroma.from_documents(
    documents=chunks, embedding=embeddings
)

# 검색: 질문과 가장 유사한 청크 3개 반환
results = vectorstore.similarity_search("회사의 주요 사업은?", k=3)
# results[0].page_content → "주요 사업 분야는 AI 솔루션 개발과..."
# results[1].page_content → "핵심 기술력으로는 자연어 처리..."
```

핵심

"주요 사업" 질문에 사업 관련 청크가 반환되고, 복지나 채용 관련 청크는 제외됩니다. k=3이면 상위 3개만 가져옵니다.

검색 결과와 질문을 LLM에 전달하기

LCEL로 검색, 프롬프트 조립, LLM 호출을 하나의 체인으로 연결합니다

```
step5_rag_chain.py python

prompt = ChatPromptTemplate.from_template("""
다음 문서를 참고하여 답변하세요.
문서에 없으면 "정보가 없습니다"라고 답하세요.
참고 문서: {context}
질문: {question}
""")

retriever = vectorstore.as_retriever(search_kwargs={"k": 3})

rag_chain = (
    {"context": retriever, "question": RunnablePassthrough()}
    | prompt | llm | StrOutputParser()
)

# rag_chain.invoke("회사의 주요 사업은?") → 문서 기반 답변
```

RAG 완성

질문이 들어오면 retriever가 관련 문서 3개를 자동 검색하고, 프롬프트에 조립하여 LLM이 근거 기반 답변을 생성합니다. 문서에 없는 내용은 "없다"고 답합니다.

체크포인트

- ✓ PDF에서 텍스트가 어떻게 추출되고, Document 객체에 어떤 정보가 담기는지 이해했습니까?
- ✓ chunk_size와 chunk_overlap이 검색 품질에 미치는 영향을 설명할 수 있습니까?
- ✓ 임베딩이 텍스트의 의미를 숫자로 변환하는 것이라는 개념을 이해했습니까?
- ✓ RAG 체인이 retriever, prompt, LLM을 파이프로 연결하는 구조를 파악했습니까?

11

RAG 챗봇 완성과 실전 예제

- ✓ RAG 파이프라인을 Streamlit 앱으로 통합한 완성형 챗봇을 이해합니다
- ✓ 일반 AI 챗봇과 RAG 챗봇의 답변 품질 차이를 체감합니다
- ✓ 다양한 챗봇 활용 사례(언어 튜터, 추천 시스템)를 체험합니다

🕒 50분

RAG 챗봇 통합 앱

Step 1~5를 하나의 Streamlit 앱(app.py)으로 통합한 최종 결과물입니다

영역	기능	구현 방식
사이드바	PDF 업로드, 처리 상태 표시, 대화 초기화	st.file_uploader, st.spinner
자동 파이프라인	업로드 시 로딩, 청킹, 임베딩, 벡터 저장을 자동 실행	session_state로 1회만 실행
채팅 영역	질문 입력, AI 스트리밍 답변, 대화 이력 유지	chat_input, write_stream
참조 문서	답변과 함께 참조한 원본 문서 조각을 표시	expander로 접기/펼치기

실행 흐름

PDF 업로드하면 자동으로 파이프라인이 실행되고, 이후 질문을 입력하면 RAG 체인이 관련 문서를 검색하여 근거 기반 답변을 스트리밍으로 출력합니다.

일반 AI vs RAG 챗봇 비교

같은 질문을 던졌을 때 답변 품질이 어떻게 달라지는지 비교합니다

비교 항목	일반 AI	RAG 챗봇
답변 내용	일반적인 답변 ("보통 IT 회사의 매출은...")	구체적인 답변 ("회사의 2024년 매출은 150억 원")
정확도	추측 기반 (틀릴 수 있음)	문서 기반 (문서 내용이 정확하면 정확)
근거 제시	없음	참조 문서와 페이지 번호 제공
환각 위험	높음 (없는 정보를 만들어냄)	낮음 (문서에 없으면 "없다"고 답변)
최신 정보	학습 시점 이후 정보 없음	업로드한 문서의 정보 반영

언어 튜터 챗봇: 시스템 프롬프트 설계

시스템 프롬프트만으로 AI의 역할을 완전히 바꿀 수 있습니다

language_tutor.py

python

```
system_prompt = """
당신은 영어 회화 튜터입니다.
```

1. 역할: 친절한 영어 회화 선생님
2. 수준: 초급~중급 학습자 대상
3. 방식:
 - 한국어 입력: 영어 번역 + 발음 팁
 - 영어 입력: 문법 교정 + 자연스러운 표현 제안
 - 매 대화 끝에 유용한 표현 1개 추가
4. 형식:
 - [번역] 또는 [교정] + [설명]
 - [보너스 표현]: 상황 - 표현 - 예문

```
"""
```

나 어제 친구 만났어

AI

[번역] "I met my friend yesterday."

[발음 팁] "met"은 "멧"이 아니라 "메트"에 가깝게 발음합니다.

[보너스 표현]

상황: 약속을 잡을 때

표현: "Let's catch up sometime!"

예문: "It's been a while! Let's catch up sometime this week."

레스토랑 추천 챗봇: 대화형 조건 수집

대화를 통해 사용자의 조건을 수집하고, 조건에 맞는 맞춤 추천을 제공하는 패턴입니다

- 챗봇: "어떤 음식점을 찾고 계신가요?"
- 사용자: "이탈리안 음식이 먹고 싶어요" (음식 종류 수집)
- 챗봇: "예산은 어느 정도 생각하고 계신가요?"
- 사용자: "1인당 3만원 정도요" (예산 수집)
- 챗봇: "위치는 어디가 편하신가요?"
- 사용자: "강남 근처요" (위치 수집, 조건 충분)
- 챗봇: 강남 이탈리안 3만원대 레스토랑 3곳 추천 (결과 제공)

설계 포인트	설명
필수/선택 구분	최소 필수 정보와 선택 정보를 구분합니다
자연스러운 수집	설문조사처럼 한꺼번에 묻지 않고 대화로 수집합니다
충분 조건 판단	필수 정보가 모이면 추천을 시작합니다
추천 형식	복수 선택지와 각각의 추천 이유를 제공합니다

오늘 배운 기술 한눈에 보기

블록 1부터 8까지, 기술이 단계적으로 쌓여서 RAG 챗봇이 완성되었습니다

블록	배운 기술	만든 것
블록 1~2	Python, OpenAI API, 메시지 유형, 싱글턴/멀티턴	터미널에서 AI와 대화
블록 3	프롬프트 엔지니어링, Zero/One/Few-shot	답변 품질을 높이는 기법
블록 4	Streamlit, LangChain, LangSmith	브라우저에서 동작하는 웹 챗봇
블록 5~6	LCEL 체인, Tool, Agent	도구를 자동 선택하는 AI Agent
블록 7	시스템 프롬프트 한계, RAG 이론, 임베딩, 벡터DB	RAG가 왜 필요한지 체감
블록 8	RAG 파이프라인, 통합 앱, 실전 예제	PDF 기반 RAG 챗봇 완성

다음 단계

오늘 배운 기초 위에 쌓을 수 있는 심화 학습 방향입니다

초급에서 중급으로

멀티 PDF RAG

여러 PDF를 동시에 검색

대화형 RAG

이전 대화를 고려한 질문 재작성

하이브리드 검색

키워드와 벡터 검색을 결합

중급에서 고급으로

LangGraph

복잡한 에이전트 워크플로우 설계

RAG 평가

답변 품질을 자동으로 측정

프로덕션 배포

Docker, 클라우드 서비스 운영

워크샵 회고

- ✓ RAG 챗봇 통합 앱의 전체 구조(사이드바, 채팅 영역, 참조 문서)를 이해했습니까?
- ✓ 일반 AI 챗봇과 RAG 챗봇의 답변 품질 차이를 체감했습니까?
- ✓ 오늘 배운 기술들(API, Streamlit, LangChain, LCEL, Tool, RAG)이 어떻게 연결되는지 설명할 수 있습니까?
- ✓ 자신의 업무에서 챗봇을 활용할 수 있는 아이디어가 있습니까?